Experiment 7

Student Name: Ashutosh Yadav **UID: 23BCS11023**
Branch: BE CSE **Section/Group: 23BCSKRG-3-A**
Semester: 6th **Date of Performance:16/03/26**
Subject Name: Full Stack Development-II **Subject Code: 23CSH-309**

1. Aim:

To implement JPA entity mapping, advanced queries, fetch strategies, and database performance optimization using Spring Boot, JPA, and Hibernate.

2. Objective: After completing this experiment, the student will be able to:

- To create JPA entity classes and map them to database tables using annotations.
- To implement One-to-Many relationships between entities using JPA.
- To perform advanced database queries using JPQL and pagination techniques.
- To analyze Lazy and Eager fetch strategies and their impact on performance.
- To optimize database performance using indexing and query tuning.

3. Implementation/Code:

Appointment.java:

```
package com.example.demo.entity;

import jakarta.persistence.*;
import lombok.*;
import java.time.LocalDateTime;
import java.util.HashSet;
import java.util.Set;

/**
 * Appointment Entity - Represents an appointment between Patient and Doctor
 * Demonstrates Many-to-One relationships and Eager fetching for core entities
 * Demonstrates N+1 problem when querying without proper fetch strategy
 */
@Entity
@Table(name = "appointment", indexes = {
    @Index(name = "idx_appointment_date", columnList = "appointment_date"),
```

```
    @Index(name = "idx_patient_id", columnList = "patient_id"),
    @Index(name = "idx_doctor_id", columnList = "doctor_id"),
    @Index(name = "idx_appointment_status", columnList = "status")
})
@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class Appointment {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    /**
     * Many-to-One with Patient
     * Using EAGER fetching for accessing patient directly helps avoid lazy loading issues
     * However, when fetching appointments, this can lead to inefficiency
     */
    @ManyToOne(fetch = FetchType.EAGER)
    @JoinColumn(name = "patient_id", nullable = false)
    private Patient patient;

    /**
     * Many-to-One with Doctor
     * EAGER fetching ensures doctor details are loaded with appointment
     * This can demonstrate the performance trade-offs
     */
    @ManyToOne(fetch = FetchType.EAGER)
    @JoinColumn(name = "doctor_id", nullable = false)
    private Doctor doctor;

    @Column(name = "appointment_date", nullable = false)
    private LocalDateTime appointmentDate;

    @Column(name = "duration_minutes")
    private Integer durationMinutes;

    @Column(length = 1000)
    private String reason;
```

```
    @Column(length = 2000)
    private String notes;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private AppointmentStatus status;

    @Column(name = "created_at")
    private LocalDateTime createdAt;

    @Column(name = "updated_at")
    private LocalDateTime updatedAt;

    /**
     * One-to-Many with Prescription
     * LAZY loading ensures prescriptions are only loaded when needed
     */
    @OneToMany(mappedBy = "appointment", fetch = FetchType.LAZY, cascade =
CascadeType.ALL, orphanRemoval = true)
    @Builder.Default
    private Set<Prescription> prescriptions = new HashSet<>();

    @PrePersist
    protected void onCreate() {
        createdAt = LocalDateTime.now();
        updatedAt = LocalDateTime.now();
        status = AppointmentStatus.SCHEDULED;
    }

    @PreUpdate
    protected void onUpdate() {
        updatedAt = LocalDateTime.now();
    }

    public enum AppointmentStatus {
        SCHEDULED, COMPLETED, CANCELLED, RESCHEDULED, NO_SHOW
    }
}
```

4. Output:

```
    /* insert
    for
        com.example.demo.entity.Prescription
        */insert
    into
        prescription (appointment_id, created_at, dosage, duration_days, expiry_date, frequency, instructions, medication_name, patient_id, prescribed_date, id)
    values
        (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, default)
Hibernate:
    /* insert
    for
        com.example.demo.entity.Prescription
        */insert
    into
        prescription (appointment_id, created_at, dosage, duration_days, expiry_date, frequency, instructions, medication_name, patient_id, prescribed_date, id)
    values
        (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, default)
2026-03-16T12:01:53.983+05:30  INFO 18276 --- [demo] [           main] com.example.demo.DataInitializer         : ? Created 3 patients
2026-03-16T12:01:53.984+05:30  INFO 18276 --- [demo] [           main] com.example.demo.DataInitializer         : ? Created 3 doctors
2026-03-16T12:01:53.985+05:30  INFO 18276 --- [demo] [           main] com.example.demo.DataInitializer         : ? Created 3 appointments
2026-03-16T12:01:53.986+05:30  INFO 18276 --- [demo] [           main] com.example.demo.DataInitializer         : ? Created 3 prescriptions
2026-03-16T12:01:53.987+05:30  INFO 18276 --- [demo] [           main] com.example.demo.DataInitializer         : ========== Sample Data Initialization Complete ==========
2026-03-16T12:01:56.017+05:30  INFO 18276 --- [demo] [on(2)-127.0.0.1] o.a.c.c.C.[Tomcat].[localhost].[/]       : Initializing Spring DispatcherServlet 'dispatcherServlet'
2026-03-16T12:01:56.018+05:30  INFO 18276 --- [demo] [on(2)-127.0.0.1] o.s.web.servlet.DispatcherServlet        : Initializing Servlet 'dispatcherServlet'
2026-03-16T12:01:56.020+05:30  INFO 18276 --- [demo] [on(2)-127.0.0.1] o.s.web.servlet.DispatcherServlet        : Completed initialization in 2 ms
```

5. Learning Outcome:

- Understand how JPA maps Java objects to relational database tables.
- Develop efficient database queries using JPQL.
- Apply pagination techniques for handling large datasets.
- Evaluate the performance impact of different fetch strategies.
- Optimize backend applications using indexing and query optimization techniques.